

PILOT POC – Build-Ready AI Coder Specification

Prepared as a build-ready handoff for a coding agent such as Codex or Claude Code.

1. Purpose

This document converts the existing PILOT AI Factory demo concept into a build-ready engineering specification suitable for use with an AI coding agent such as OpenAI Codex or Claude Code.

The objective is to build a deterministic demonstration web application that shows the following proposition:

Digital actions can create physical consequences.

Reported telemetry can be falsified so that correlational tools miss the issue.

PILOT detects the causal inconsistency by comparing inferred physical state, reported telemetry, and cross-zone behaviour.

The demo is intentionally simplified. It is not intended to be a production thermal simulator. It is intended to be a clear, deterministic proof-of-concept that demonstrates the difference between correlation and feasibility validation.

2. Required Demo Behaviour

The application must:

1. Load one or more YAML scenario files.
2. Validate the scenario against a formal schema.
3. Simulate baseline thermal behaviour under normal conditions.
4. Simulate actual thermal behaviour under the scenario inputs.
5. Optionally apply telemetry falsification to reported telemetry.
6. Apply a relevance filter so that only state-defining or state-changing data is used.
7. Run PILOT causal checks against the scenario.
8. Run Sigma-style correlational rules in parallel against the reported telemetry.
9. Produce a side-by-side result showing:
 - PILOT outcome: valid, violation, or uncertain
 - Sigma outcome: alert/no alert or triggered rules
 - explanation of why the outcomes differ
10. Present results through:
 - a command-line interface for local testing
 - a simple web application for demonstration use

3. Core Demonstration Logic

The demo must support at least four scenario classes:

A. Valid scenario

A legitimate digital load increase causes a plausible temperature rise and plausible neighbour effects.

Expected outcome:

- PILOT: valid
- Sigma: may trigger informational or no alert, but no contradiction

B. Operational issue

A non-malicious issue such as insufficient cooling causes overheating.

Expected outcome:

- PILOT: valid or violation depending on the chosen rule framing, but the system must clearly distinguish "physically consistent fault" from "causally impossible state"
- Sigma: likely alerts on high temperature

For the first implementation, treat operational overheating without falsified telemetry as:

- PILOT status: valid
- Operational flag: issue_detected

This preserves the distinction that PILOT is validating feasibility, not merely saying whether the state is desirable.

C. Falsified telemetry

The actual thermal state is inconsistent with the reported telemetry because reported temperatures are suppressed, flattened, or otherwise manipulated.

Expected outcome:

- PILOT: violation
- Sigma: likely misses or understates the issue

D. Incomplete data

Required telemetry is missing or insufficient to resolve the state.

Expected outcome:

- PILOT: uncertain
- Sigma: whatever reported rules produce

4. Functional Scope

The first build must include:

- Python backend
- YAML scenario ingestion
- Deterministic thermal simulation
- Telemetry falsification engine
- PILOT rules engine
- Sigma-style rules engine
- Comparison report generator
- REST API
- Minimal web UI

- Automated tests

The first build does not need:

- Real CFD
- GPU integration
- Real SIEM integration
- Identity provider integration
- Multi-user tenancy
- Streaming telemetry
- Persistent historical analytics
- Complex auth
- Real digital twin platform integration

5. Recommended Tech Stack

Backend

- Python 3.11+
- FastAPI for API layer
- Pydantic for schema validation
- PyYAML for YAML parsing
- Uvicorn for local serving

Frontend

Either of the following is acceptable:

- Simple server-rendered Jinja templates
- React frontend calling the FastAPI backend

For the first build, the simplest acceptable implementation is:

- FastAPI backend
- Jinja HTML templates
- basic CSS
- no frontend framework required

Testing

- pytest
- coverage optional

Packaging

- Dockerfile
- docker-compose.yml for local development

6. Repository Structure

The repository should be created with the following structure:

```
pilot-poc/  
README.md  
AGENTS.md  
requirements.txt  
Dockerfile  
docker-compose.yml  
.env.example  
app/  
main.py  
config.py  
api/  
routes_scenarios.py  
routes_simulation.py  
routes_reports.py  
models/  
scenario_models.py  
output_models.py  
services/  
scenario_loader.py  
thermal_model.py  
falsification_engine.py  
relevance_filter.py  
pilot_engine.py  
sigma_engine.py  
comparison_engine.py  
report_builder.py  
templates/  
index.html  
scenario_result.html  
static/  
styles.css  
schemas/  
scenario.schema.json  
scenarios/  
valid_normal.yaml  
operational_overheat.yaml  
falsified_telemetry.yaml  
incomplete_data.yaml
```

tests/
test_schema_validation.py
test_thermal_model.py
test_falsification_engine.py
test_pilot_engine.py
test_sigma_engine.py
test_end_to_end.py
docs/
SPEC.md
RULES.md
DEPLOYMENT.md

7. AGENTS.md for Codex or Claude

The repository must include an AGENTS.md file with instructions similar to the following:

- Build the application in small, testable steps.
- Preserve deterministic behaviour.
- Do not introduce machine learning.
- Prefer explicit rule logic over clever abstractions.
- Write tests before or alongside implementation where practical.
- Keep the simulation simple and explainable.
- Do not replace PILOT feasibility logic with anomaly detection.
- All outputs must include human-readable explanations.
- If there is uncertainty due to missing required telemetry, return uncertain rather than guessing.
- Keep the UI minimal and functional.

8. Formal Scenario Schema

Each scenario YAML must follow this conceptual schema.

Top level

- scenario
- digital
- physical
- telemetry
- baseline
- pilot
- sigma
- metadata optional

Required fields

scenario

- name: string
- description: string
- duration_minutes: integer > 0
- timestep_seconds: integer > 0
- zones: array of zone identifiers, minimum 1

digital

- gpu_load: mapping of zone -> float 0.0 to 1.0
- scheduler_events: optional list of timed digital changes

physical

- cooling_capacity: mapping of zone -> float 0.0 to 1.0
- ambient_temp_c: float
- neighbour_heat_bleed: float 0.0 to 1.0
- thermal_inertia: float > 0
- sensor_noise_c: float >= 0

telemetry

- falsification:
- enabled: boolean
- zones: list of zones to manipulate
- reported_temp_offset_c: float
- flatten_trend: boolean
- dropout_probability: float 0.0 to 1.0
- missing_fields: optional list

baseline

- expected_temp_at_load:
mapping of load band string to [min_c, max_c]
- neighbour_response_expected: boolean
- max_reporting_delta_c: float
- max_cross_zone_delta_c: float
- min_required_zones_reporting: integer

pilot

- checks: list of check names
- thresholds:
- reported_vs_inferred_delta_c
- cross_zone_delta_c
- sustained_violation_minutes

- neighbour_response_lag_minutes

sigma

- rules:

- name

- expression

- level

9. Example Build-Ready YAML

```
scenario:
  name: asymmetric_gpu_load_with_falsified_telemetry
  description: High load in zone A creates real heat that is under-reported in telemetry.
  duration_minutes: 20
  timestep_seconds: 60
  zones: [A, B, C]

digital:
  gpu_load:
    A: 0.92
    B: 0.35
    C: 0.30

physical:
  cooling_capacity:
    A: 0.60
    B: 0.60
    C: 0.60
  ambient_temp_c: 24
  neighbour_heat_bleed: 0.08
  thermal_inertia: 0.85
  sensor_noise_c: 0.2

telemetry:
  falsification:
    enabled: true
    zones: [A]
    reported_temp_offset_c: -9
    flatten_trend: true
    dropout_probability: 0.0
    missing_fields: []

baseline:
  expected_temp_at_load:
    "0.30": [36, 38]
    "0.60": [42, 45]
    "0.90": [48, 54]
  neighbour_response_expected: true
  max_reporting_delta_c: 4.0
  max_cross_zone_delta_c: 10.0
  min_required_zones_reporting: 3

pilot:
  checks:
    - thermal_inconsistency
    - cross_zone_inconsistency
    - reported_vs_inferred
  thresholds:
    reported_vs_inferred_delta_c: 4.0
    cross_zone_delta_c: 10.0
    sustained_violation_minutes: 3
    neighbour_response_lag_minutes: 4

sigma:
  rules:
    - name: High Temperature Alert
      expression: "temperature > 50"
      level: high
    - name: Load and Temperature Correlation
      expression: "(gpu_load > 0.8) and (temperature > 48)"
```

```
level: medium
- name: Zone Imbalance Detection
  expression: "abs(temp_A - temp_B) > 10"
level: medium
```

10. Thermal Model Specification

The thermal model must be deliberately simple and deterministic.

For each zone z at each timestep t :

1. Start from previous temperature $T[z, t-1]$
2. Add heat from local GPU load
3. Subtract effect of cooling capacity
4. Add neighbour bleed from adjacent zones
5. Apply inertia so that temperatures change smoothly
6. Add bounded sensor noise only to actual sensed values, not to inferred model truth

A simple acceptable implementation is:

```
target_temp[z] =
ambient_temp_c
+ load_factor * gpu_load[z]
- cooling_factor * cooling_capacity[z]
+ neighbour_heat_bleed * mean(positive_neighbour_excess_heat)
```

Then:

$$T[z, t] = \text{thermal_inertia} * T[z, t-1] + (1 - \text{thermal_inertia}) * \text{target_temp}[z]$$

The constants should be chosen so that the simulated temperatures broadly match the baseline ranges:

- 30 percent load -> 36 to 38 C
- 60 percent load -> 42 to 45 C
- 90 percent load -> 48 to 54 C

The coder may calibrate constants, but the resulting outputs must satisfy the test cases.

11. Telemetry Types

The system must maintain three distinct telemetry views:

1. Baseline telemetry

Expected normal ranges and normal trend behaviour.

2. Actual telemetry

The simulated actual physical state.

3. Reported telemetry

The telemetry exposed to the Sigma rules and UI after any falsification is applied.

This distinction is essential and must not be collapsed.

12. Relevance Filter

Only data that defines or changes state should be retained for PILOT reasoning.

For the AI Factory demo, retain:

- zone identifier
- timestamp
- gpu_load
- cooling_capacity
- actual_temp
- reported_temp
- neighbour relationships
- missing-data indicators

Discard any data that does not affect thermal state or causal interpretation.

The relevance filter can be simple in version one, but it must exist as an explicit step in the pipeline.

13. PILOT Rule Definitions

The following rules must be implemented explicitly.

Rule 1: thermal_inconsistency

Purpose:

Determine whether the actual or inferred thermal state is implausible given digital load, cooling, and neighbour effects.

Inputs:

- gpu_load by zone
- cooling_capacity by zone
- inferred temperature by zone
- baseline expected ranges

Logic:

- infer expected temperature band for each zone from load and cooling
- if inferred or actual state lies materially outside expected bounds without an allowed explanation, mark violation candidate

Version one interpretation:

- this rule primarily supports explanation and sanity checking
- use a tolerance band of plus/minus 3 C around interpolated expected temperature

Rule 2: reported_vs_inferred

Purpose:

Determine whether reported telemetry is inconsistent with the inferred or actual state.

Inputs:

- reported temperature
- inferred temperature
- threshold

Logic:

- for each zone, compute absolute difference between reported and inferred temperature
- if difference exceeds threshold for at least sustained_violation_minutes, trigger violation

Default threshold:

- 4.0 C sustained for 3 minutes

Rule 3: cross_zone_inconsistency

Purpose:

Determine whether neighbouring zones show expected thermal interaction.

Inputs:

- actual or inferred zone temperatures
- zone adjacency
- neighbour heat bleed expectation

Logic:

- when one zone has sustained high load and high inferred temperature, neighbouring zones should show some delayed rise
- if reported data suppresses local heat but neighbours still rise, this creates contradiction
- if expected neighbour response is absent from both actual and reported data, no contradiction is inferred from this rule alone

Rule 4: insufficient_data

Purpose:

Return uncertain when the model lacks required telemetry.

Inputs:

- reporting coverage
- required field presence

Logic:

- if fewer than min_required_zones_reporting zones are available, or if mandatory temperature values are missing for sustained_violation_minutes, return uncertain

14. Outcome Logic

The system must output one primary PILOT state:

- valid
- violation
- uncertain

Decision order:

1. If insufficient_data applies, return uncertain.
2. Else if reported_vs_inferred or cross_zone_inconsistency is triggered, return violation.
3. Else return valid.

Operational issues such as overheating without falsification should be surfaced separately as operational findings, not confused with causal impossibility.

Therefore a report may contain:

- pilot_status: valid
- operational_findings: ["zone A overheating"]

15. Sigma Engine Specification

The Sigma engine is not a full Sigma implementation.

It is a lightweight correlational comparator that evaluates simple boolean rules over reported telemetry only.

Requirements:

- parse rule definitions from YAML or internal config
- evaluate against reported telemetry only
- return:
 - triggered_rules
 - highest_level
 - alert boolean

This comparator is intentionally limited. Its purpose is to show what a correlational layer sees when fed manipulated telemetry.

16. API Specification

The backend must expose at least the following routes.

GET /

Return landing page with scenario upload and scenario selection.

POST /api/scenarios/validate

Input:

- YAML file or YAML text

Output:

- schema validation result
- error messages if invalid

POST /api/simulate

Input:

- scenario

Output:

- baseline telemetry
- actual telemetry
- reported telemetry

POST /api/evaluate

Input:

- scenario

Output:

- pilot result
- sigma result
- explanations
- evidence

POST /api/run

Input:

- scenario

Output:

Complete end-to-end result in one response.

GET /api/health

Return simple health response.

17. Output Contract

The end-to-end run must return JSON in this shape:

```
{
  "scenario_name": "asymmetric_gpu_load_with_falsified_telemetry",
  "pilot_status": "violation",
  "operational_findings": [],
  "sigma": {
    "alert": false,
    "highest_level": null,
    "triggered_rules": []
  },
  "explanations": [
    "Reported temperature in zone A is materially below inferred temperature for 3 consecutive minutes.",
    "Neighbouring zones indicate heat propagation inconsistent with the reported flat trend in zone A."
  ],
  "evidence": {
    "zone_A": {
      "gpu_load": 0.92,
      "inferred_peak_temp_c": 52.1,
      "reported_peak_temp_c": 43.0,
      "max_reported_vs_inferred_delta_c": 9.1
    }
  }
}
```

```
    }  
  },  
  "timesteps": 20  
}
```

18. Web UI Requirements

The web UI should be minimal and demo-oriented.

It must allow the user to:

- upload a YAML file
- choose from built-in sample scenarios
- run the scenario
- view:
 - scenario summary
 - key charts
 - PILOT result
 - Sigma result
 - explanation text
 - evidence summary

Recommended charts:

- actual temperature by zone over time
- reported temperature by zone over time
- inferred versus reported for selected zone
- simple result panel showing PILOT versus Sigma

For version one, charts can be rendered with:

- Chart.js

or

- Plotly

19. Test Cases

At minimum, include the following automated tests.

Test 1: schema validation passes for valid scenario

Expected:

- no validation errors

Test 2: schema validation fails for invalid scenario

Expected:

- missing required fields produce clear errors

Test 3: valid normal scenario returns valid

Expected:

- PILOT valid
- no contradiction explanations

Test 4: operational overheating without falsification

Expected:

- PILOT valid
- operational finding present
- Sigma may alert

Test 5: falsified telemetry returns violation

Expected:

- PILOT violation
- Sigma alert may be absent or weaker
- explanation includes reported_vs_inferred inconsistency

Test 6: missing required telemetry returns uncertain

Expected:

- PILOT uncertain

Test 7: neighbour contradiction contributes to violation

Expected:

- PILOT violation due to cross-zone inconsistency

20. Acceptance Criteria

The build is acceptable only if all of the following are true:

1. A user can upload or select a YAML scenario and run it through the app.
2. The app clearly separates baseline, actual, and reported telemetry.
3. The app clearly shows the PILOT result and Sigma result side by side.
4. The falsified telemetry scenario produces a PILOT violation.
5. The incomplete data scenario produces uncertain.
6. The output includes human-readable explanation text.
7. The codebase includes tests and the tests pass.
8. The app runs locally via Docker and without Docker.
9. The logic remains deterministic and explainable.
10. No machine learning is introduced into the core reasoning path.

21. What to Give Codex or Claude

To get good results from Codex or Claude, upload or provide:

1. This specification

2. The existing concept PDF
3. The sample YAML scenarios
4. The expected JSON output examples
5. Clear instruction to implement in phases:
 - phase 1: backend schema and simulator
 - phase 2: PILOT and Sigma engines
 - phase 3: web UI
 - phase 4: tests and Docker packaging

Recommended prompt to the coding agent:

Build this repository exactly as specified.

Start with the schema models, YAML loader, thermal simulator, falsification engine, and automated tests.

Then implement the PILOT engine and Sigma comparator.

Then expose a FastAPI web app with a minimal UI.

Do not introduce ML.

Keep the code explicit, simple, and heavily testable.

Where thresholds are not fully defined, implement the defaults in this spec and keep them centrally configurable.

Return progress in small commits or clear file-group batches.

22. Online Resources Needed to Run This as a Web App

For a basic demo web app, the required online resources are modest.

Smallest practical hosted setup

You need:

1. One application host

This runs the FastAPI web app and serves the UI.

For a demo, a single small virtual machine or container instance is enough.

2. Optional reverse proxy

Nginx or equivalent in front of the app for TLS termination and routing.

Some hosting platforms handle this for you automatically.

3. Domain name

Optional for internal use, but recommended if you want a stable public URL.

4. TLS certificate

Needed for HTTPS if exposed on the internet.

Often automated by the hosting provider or via Let's Encrypt.

5. Optional persistent storage

Not required if scenarios are uploaded and processed in-memory only.

Required if you want saved scenarios, uploaded files, user accounts, or audit history.

6. Optional database

Not required for a single-user or stateless demo.

Useful only if you want:

- stored scenarios
- saved runs
- user management
- audit logs
- multi-session history

7. Monitoring/logging

At minimum:

- app logs
- error logs

Nice to have:

- uptime monitoring
- request metrics

Resource sizing for a basic demo

For a lightweight demo with no real CFD and no heavy analytics:

Minimum reasonable starting point:

- 1 CPU
- 1 to 2 GB RAM
- 10 to 20 GB storage

More comfortable demo setup:

- 2 CPU
- 2 to 4 GB RAM
- 20 GB storage

This is enough because the application is mostly:

- YAML parsing
- lightweight simulation
- rule evaluation
- chart rendering

It does not require GPU resources.

Recommended deployment shapes

Option A: simplest

Single container or single VM:

- FastAPI app
- server-rendered UI
- no database

Best for:

- internal demo
- cheapest path
- fastest deployment

Option B: slightly more robust

- FastAPI app container
- reverse proxy
- optional Postgres database

Best for:

- repeatable hosted demo
- small internal team use
- scenario persistence

Option C: polished customer demo

- app container
- reverse proxy
- database
- object storage for uploaded scenarios and reports
- basic authentication
- monitoring

Best for:

- external customer demo
- multiple demo users
- retained evidence and reports

23. Security and Operational Notes

If the app is internet-exposed, implement at least:

- HTTPS
- file upload size limits
- YAML parsing safeguards
- basic rate limiting
- input validation
- no arbitrary code execution
- no shell execution from user inputs

- secrets via environment variables
- debug mode disabled in production

For a customer-facing demo, add:

- basic authentication
- audit logging
- admin-only upload rights
- malware scanning for uploaded files if arbitrary file types are ever allowed

For version one, only YAML uploads should be accepted.

24. Recommended Build Sequence

The coding agent should implement in this order:

1. project skeleton
2. pydantic models
3. YAML schema validation
4. thermal simulation
5. falsification engine
6. relevance filter
7. PILOT engine
8. Sigma engine
9. report builder
10. API routes
11. minimal UI
12. Docker packaging
13. automated tests
14. polish and documentation

25. Bottom Line

What you need is not just a concept PDF.

You need a structured coder pack that removes ambiguity.

This document is intended to be that pack.

Used together with the original demo PDF, it should be sufficient for a capable coding agent such as Codex or Claude Code to build a first working version of the PILOT AI Factory demo as a web application.

The correct first goal is not production realism.

The correct first goal is a deterministic, explainable, testable demonstration that proves the distinction between correlation and causal feasibility validation.